

# Foundations of Deep Learning—Written Exam

Master’s Degree in AI & Society (PSL-PSAI, Spring 2026)

13 May 2026—Solutions

## Part 1—True / False

**1.1—TRUE.** Without hidden layers, the network computes  $y = h(W\mathbf{x} + \mathbf{b})$ , where  $h$  is the output activation. The decision boundary is determined by where the output changes class—i.e. where the pre-activation  $W\mathbf{x} + \mathbf{b}$  crosses a threshold, which is always a hyperplane in the input space regardless of  $h$ . For example, with a scalar input and tanh output:

$$y = \tanh(wx + b)$$

the decision boundary is where  $y = 0$ , i.e.  $\tanh(wx + b) = 0$ , which reduces to  $wx + b = 0$ , or  $x = -b/w$ , a single point on the real line. The tanh only squashes the output into  $(-1, 1)$ ; it does not bend the decision boundary. This is why a single-layer network cannot solve XOR, which requires a non-linear boundary. Hidden layers with non-linear activations transform the input representation into a space where the data may become linearly separable, giving the network the capacity to learn curved decision boundaries.

**1.2—FALSE.** The universal approximation theorem is a statement about *width*, not depth. It states that a single hidden layer with a sufficiently large number of neurons can approximate any continuous function on a compact domain, provided the activation function is non-linear. A single hidden layer is therefore sufficient—depth plays no role in the theorem. What depth provides in practice is *efficiency*: deep networks can represent certain functions with exponentially fewer parameters than a single very wide hidden layer.

**1.3—TRUE.** Since  $\mathcal{L}^{\text{MSE}} = \mathcal{L}^{\text{SSE}}/N$ , their gradients are related by:

$$\nabla_{\mathbf{w}} \mathcal{L}^{\text{MSE}} = \frac{1}{N} \nabla_{\mathbf{w}} \mathcal{L}^{\text{SSE}}$$

The gradient descent update with MSE and learning rate  $\eta$  is:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}^{\text{MSE}} = \mathbf{w} - \frac{\eta}{N} \nabla_{\mathbf{w}} \mathcal{L}^{\text{SSE}}$$

which is identical to the update using SSE with learning rate  $\eta/N$ . The two losses are therefore interchangeable provided the learning rate is scaled accordingly—the choice between them is a matter of convention, not substance.

**1.4—FALSE.** Backpropagation requires only that each operation in the computational graph has a well-defined local gradient—it makes no assumptions about which loss function or output activation is used. Any differentiable combination of output activation and loss can be used with backpropagation, including non-canonical ones such as sigmoid + MSE. The canonical pairs are preferred not because they are necessary for backpropagation, but because they produce simpler and numerically more stable gradients: for example, softmax + cross-entropy gives the clean gradient  $\mathbf{y} - \mathbf{t}$  at the output layer, avoiding the saturation issues that arise with non-canonical combinations.

**1.5—FALSE.** Standard CNNs are equivariant to *translations* but not to arbitrary rotations. Rotation equivariance does not hold because filters are fixed in orientation—a rotated version of a feature will not necessarily activate the same filter. For example, a filter trained to detect a horizontal edge will not respond to the same edge rotated by 90 degrees. Achieving rotation equivariance requires either specialised architectures (e.g. group-equivariant CNNs) or data augmentation with rotated examples, which teaches the network to be approximately invariant to rotations through exposure rather than by design.

**1.6—TRUE.** When the kernel size equals the full spatial dimensions of the input, each filter performs a dot product over the entire  $H \times W \times C_{in}$  volume, producing a single scalar output. With  $C_{out}$  such filters the output has shape  $1 \times 1 \times C_{out}$ , which is equivalent to a vector of  $C_{out}$  values. This is identical to what a fully connected layer does: it computes  $C_{out}$  independent linear combinations of all  $H \times W \times C_{in}$  inputs. The number of weight parameters is the same in both cases:  $(HWC_{in} + 1)C_{out}$ . This equivalence is why fully connected layers can be replaced by convolutional layers in fully-convolutional architectures, enabling the network to process inputs of arbitrary spatial size (see OverFeat for an example).

**1.7—TRUE.** The MLP processes each token representation independently—it is applied to each row of  $Z$  separately, with no interaction between positions. This is what “position-wise” means. Context is handled exclusively by the self attention sub-block, which aggregates information from all positions through the attention weights. The MLP’s role is complementary: it introduces a non-linear transformation at each position independently, breaking the constraint that attention outputs are linear combinations of the value vectors and allowing richer token-level representations to be learned.

**1.8—FALSE.** High accuracy on object recognition benchmarks does not imply that a model has acquired objectness in the sense that human infants possess it. Infants develop objectness through sensorimotor interaction with the physical world — they learn that objects persist when hidden, resist penetration, and move as cohesive wholes through direct physical experience. AI systems, by contrast, learn statistical associations between pixel patterns and labels from curated datasets. A model that correctly labels a dog in a photograph has no understanding that the dog is a persistent, bounded entity that continues to exist when out of frame—it has learned a mapping from visual patterns to category names.

## Part 2—Multiple Choice

**2.1—Correct: A, B, D.**

**(A) True.** This is the key property of canonical pairs: the combined gradient through the output activation and loss simplifies to  $\mathbf{y} - \mathbf{t}$  for all three pairs. This clean form avoids the saturation issues

that arise with non-canonical combinations and makes the backward pass numerically stable.

**(B) True.** SSE corresponds to MLE under Gaussian noise ( $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ ); BCE corresponds to MLE under a Bernoulli distribution; cross-entropy corresponds to MLE under a categorical distribution. The canonical pairs are not arbitrary—they each have a principled probabilistic justification.

**(C) False.** For multi-label classification, softmax is inappropriate because it forces the outputs to sum to 1, implying mutual exclusivity of classes. The correct choice is independent sigmoid activations on each output unit combined with binary cross-entropy, allowing multiple classes to be simultaneously active.

**(D) True.** Continuous regression targets can take any value in  $\mathbb{R}$ , so the output activation must be unbounded. The identity function  $y = a$  achieves this. Bounded activations such as sigmoid or tanh would prevent the network from predicting values outside their range.

**(E) False.** Cross-entropy and classification accuracy are different objectives with different minimisers. Accuracy is a step function of the logits—it changes only when the predicted class flips—and has zero gradient almost everywhere. Cross-entropy is used as a smooth, differentiable *proxy* for accuracy that is amenable to gradient-based optimisation. A model that minimises cross-entropy will generally perform well on accuracy, but the two are not equivalent and are not minimised by the same parameters in general.

## 2.2—Correct: C,D,E

**(A) False.** As the number of parameters grows, a critical point requires *all* eigenvalues of the Hessian to be positive to be a local minimum—an increasingly unlikely condition. Most critical points in high-dimensional spaces are therefore saddle points, not local minima.

**(B) False.** In high-dimensional NNs, local minima tend to concentrate near the global minimum in terms of loss value. The risk of getting trapped in a poor local minimum is therefore less critical in practice for large networks than low-dimensional intuition would suggest.

**(C) True.** The gradient noise introduced by random mini-batches provides perturbations that help escape saddle points and flat regions where the deterministic gradient would be near zero. Stochasticity is not merely a computational convenience but has a beneficial effect on optimisation dynamics.

**(D) True.** In a loss landscape with stiff and soft directions, the optimal learning rate for the soft direction is large (low curvature) while the optimal for the stiff direction is small (high curvature). A single fixed learning rate cannot satisfy both, leading to oscillations in stiff directions — the characteristic zigzag behaviour of vanilla SGD.

**(E) True.** The Hessian eigenvalues measure curvature along the corresponding eigenvectors. All positive eigenvalues imply a local minimum. All negative imply a local maximum. Mixed signs imply saddle point (the loss decreases along some directions and increases along others).

## 2.3—Correct: A,B

**(A) True.** The regularised loss can be written as  $\tilde{\mathcal{L}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \lambda \|\mathbf{w}\|^2$ . The penalty term  $\lambda \|\mathbf{w}\|^2 = -\log p(\mathbf{w}) + \text{const}$  corresponds to the negative log-likelihood of a Gaussian distribution over the weights. Minimising  $\tilde{\mathcal{L}}$  therefore jointly maximises the data likelihood and the likelihood of the weights under this Gaussian, giving L2 regularisation a direct probabilistic interpretation.

**(B) True.** The gradient is  $\nabla \tilde{\mathcal{L}}(\mathbf{w}) = \nabla \mathcal{L}(\mathbf{w}) + 2\lambda \mathbf{w}$ . The additional term  $2\lambda \mathbf{w}$  shrinks each weight at every step regardless of the data—hence the name *weight decay*.

**(C) False.** The L2 penalty  $\lambda \|\mathbf{w}\|^2$  is *isotropic*: it adds the same bowl-shaped curvature  $2\lambda$  in every direction without distinction. It does not selectively target soft directions. The effect is that soft directions (low original curvature) gain relatively more from the added  $2\lambda$  than stiff ones (high original curvature), which does make the contours more uniform—but this is a side effect of isotropy, not a selective mechanism.

**(D) False.** Increasing  $\lambda$  does not always improve generalisation. For  $\lambda \rightarrow \infty$ , all weights are driven to zero, producing a trivial model that ignores the data. The optimal  $\lambda$  balances the data fit and the regularisation penalty, and must be selected via validation performance. Too large a  $\lambda$  causes underfitting.

**(E) False.** In the limit  $\lambda \rightarrow \infty$ , the penalty dominates and drives all weights towards zero—the model becomes trivial, not maximally expressive. It is  $\lambda \rightarrow 0$  that recovers the full, unregularised model. The regularisation strength  $\lambda$  tunes the *effective* number of parameters: large  $\lambda$  means fewer effective parameters, small  $\lambda$  means more.

## 2.4—Correct: B,C,D

**(A) False.** The output is a tensor of shape  $H \times W \times C$ , not  $H \times W$ . Each spatial position contains a vector of  $C$  class probabilities (after softmax), not a single class index. A hard class assignment of shape  $H \times W$  is only obtained by taking the argmax of the output tensor at inference time.

**(B) True.** Since each pixel is an independent classification problem, the total loss is the sum of per-pixel cross-entropy losses. This is exactly the training objective shown in the lecture for the encoder-decoder segmentation architecture.

**(C) True.** Skip connections in U-Net concatenate feature maps from encoder layers directly to the corresponding decoder layers at the same resolution. This allows the decoder to access fine-grained spatial details (edges, textures) that would otherwise be lost at the bottleneck, improving localisation accuracy.

**(D) True.** The encoder progressively downsamples (decreasing  $H$  and  $W$ ) while increasing semantic abstraction, and the decoder progressively upsamples to recover the spatial resolution needed for pixel-level predictions. Transpose convolution is one of the standard upsampling operations used in the decoder.

**(E) False.** Transpose convolutions learn a parametric upsampling but do not recover the exact input to the corresponding encoder convolution—they are not mathematical inverses of the encoder operations. Information is irreversibly lost during downsampling (e.g. through pooling or strided convolutions). This is precisely why skip connections in U-Net are necessary: they provide the decoder with direct access to the high-resolution encoder features that cannot be recovered by upsampling alone.

## 2.5—Correct: B, C

**(A) False.** An autoregressive language model generates text *sequentially*, one token at a time: at each step it samples the next token conditioned on all previously generated tokens, then feeds it back as input for the next step. Tokens cannot be generated simultaneously since each depends on all previous ones.

**(B) True.** An  $n$ -gram model approximates  $p(\mathbf{x}_1, \dots, \mathbf{x}_N) \simeq \prod_{n=1}^N p(\mathbf{x}_n | \mathbf{x}_{n-1}, \dots, \mathbf{x}_{n-L})$ . The number of distinct histories of length  $L$  over a vocabulary  $V$  is  $V^L$ , growing exponentially — making  $n$ -grams infeasible for realistic context sizes.

**(C) True.** Parameter sharing across time steps is the key architectural property of RNNs that enables variable-length processing and parameter efficiency. The same weight matrices are applied at every step, so the number of parameters does not grow with sequence length—unlike a feed-forward model that would need a separate weight for each position.

**(D) False.** RNNs are inherently sequential: each hidden state  $\mathbf{z}_n$  depends on the previous state  $\mathbf{z}_{n-1}$ , so time steps cannot be parallelised. This is one of the key limitations listed in the lecture: impossible parallelisation.

**(E) False.** The bottleneck problem and the vanishing gradient are two distinct limitations of RNNs. The bottleneck problem refers specifically to the difficulty of encoding an entire input sequence of arbitrary length into a single fixed-size hidden vector—an information compression problem. Vanishing gradients are a separate training difficulty caused by the multiplicative chain of Jacobians over time steps.

## 2.6—Correct: A,B,E

**(A) True.** Self-attention computes  $\text{softmax}(QK^\top / \sqrt{D_k})V$ , which treats all positions symmetrically—permuting the input tokens produces the same permutation of output representations. Positional encodings break this symmetry by injecting position-dependent information into the token representations before the attention is computed.

**(B) True.** Sinusoidal positional encodings use fixed trigonometric functions of position and dimension:  $r_{n,i} = \sin(n/10000^{i/D})$ ,  $r_{n,i} = \cos(n/10000^{(i-1)/D})$ . Since these are defined for any integer  $n$ , the encoding extends naturally to positions beyond the maximum sequence length seen during training. This is one of the key practical advantages of sinusoidal over learned positional encodings, which are discrete lookup tables with no encoding for unseen positions.

**(C) False.** Positional encodings are added to the token embeddings *before* the transformer layers, not after. Applying them after would mean the attention mechanism never has access to positional information when computing its representations—defeating the purpose entirely.

**(D) False.** Even without positional encodings, tokens remain distinguishable through both their content (different token embeddings) and their context (different interactions through self-attention). What is lost is only positional information: the model can no longer determine the order or absolute position of tokens in the sequence. As a result, different tokens can still produce different output representations, even though the model is insensitive to permutations of the input sequence.

**(E) True.** Since positional encodings are added element-wise to the token embeddings ( $\tilde{\mathbf{x}}_n = \mathbf{x}_n + \mathbf{r}_n$ ), they must have the same dimension  $D$  as the token embeddings. This is a necessary architectural constraint.

## Part 3—Short Answers

*Note: these are just examples of answers, others may be accepted.*

### 3.1 Generalisation in Deep Learning

(a) Bias measures how far the model’s average prediction is from the true value—a high-bias model makes systematic errors because it is too simple to capture the structure of the data. Variance measures how sensitive the model’s predictions are to the specific training set—a high-variance model fluctuates widely across different samples and fits noise as well as signal. These two sources of error trade off: underfitting occurs when bias dominates and the model performs poorly on both training and test data; overfitting occurs when variance dominates and training error is low but the model fails to generalise to unseen data.

(b) Hyperparameters (such as the number of layers, parameters etc) cannot be selected using the training set alone—a more complex model will always fit the training data better, giving no signal about generalisation. Instead, a *validation set*—data held out from training—is used to measure error on unseen examples. Hyperparameters are selected by comparing validation error across candidate configurations and choosing the one that minimises it; a rising validation error while training error keeps decreasing is the signal that the model is overfitting. The classical picture then predicts a U-shaped validation error curve as a function of model complexity: high for very simple models (underfitting), decreasing to an optimum, then increasing again as the model overfits.

(c) Modern deep learning violates the classical prediction through the *double descent* phenomenon: beyond the *interpolation threshold*—the point where the model is just large enough to perfectly fit the training data—generalisation error does not remain high but instead decreases again as the model becomes increasingly overparametrised. A model can therefore simultaneously memorise the training data and generalise well, which the classical U-shaped picture declares impossible.

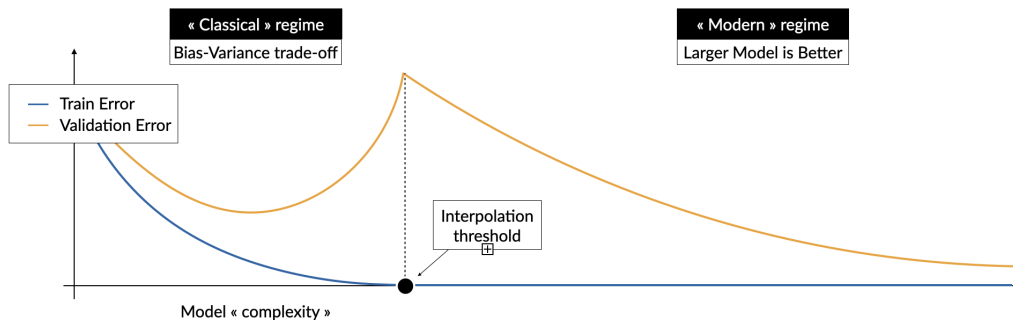


Figure 1: Double Descent, FRM S21, L6.

### 3.2 Transfer Learning

(a) The two strategies are:

- *Feature extraction*: freeze all pretrained weights and replace only the final classification layer, which is trained from scratch on the target task. The pretrained network acts as a fixed feature extractor—no gradient flows into the convolutional layers.
- *Fine-tuning*: initialise the network with pretrained weights and retrain the entire network (or a large portion of it) on the target task with a small learning rate. All parameters are updated, but they start from a useful initialisation rather than random noise. The key difference is which parameters receive gradient updates: feature extraction updates only the new classifier head, while fine-tuning updates all (or most) weights.

(b)

**Task A** (500 images): feature extraction is recommended. With only 500 examples, fine-tuning ResNet-50’s millions of parameters would cause severe overfitting—the model has far more degrees of freedom than the data can constrain. Freezing the backbone reduces trainable parameters to just the classifier head. The domain gap is manageable: both ImageNet and dermatological images share similar textures and scales, so pretrained features are useful. This is exactly the approach of Esteva et al. (Nature, 2017), who achieved dermatologist-level accuracy this way.

**Task B** (80,000 images): fine-tuning is recommended. The larger dataset provides enough supervision to update the pretrained weights without overfitting. Satellite imagery also differs substantially from ImageNet photographs — top-down viewpoint, different textures, different object scales — so the later task-specific layers need to adapt. Fine-tuning allows this adaptation while still benefiting from low-level features learned on ImageNet as a starting point.

(c) CNNs learn hierarchically: early layers detect generic low-level features (edges, colour gradients, simple textures) that are useful across virtually any visual task regardless of domain. Later layers encode increasingly task-specific abstractions (object parts, categories) tightly coupled to the source dataset. Since edges and textures appear in medical images and satellite photos just as in natural photographs, early features transfer broadly. Later features, however, encode ImageNet-specific concepts with no counterpart in the target domain, and reusing them directly would hurt performance. This is why fine-tuning typically keeps early layers close to their pretrained values and allows later layers to adapt more substantially.

## Part 4—Exercises

### 4.1 Encoder-Decoder Architecture

(a) CONV-OUT has **4 filters** and output shape  $(32, 32, 4)$ .

The number of filters (4) equals the number of semantic classes  $C$ —this is known in advance because each output channel must provide a class score for every pixel. The spatial dimensions  $(32 \times 32)$  must match the input image so that each pixel in the output corresponds to exactly one pixel in the input. For segmentation, both the number of channels and the spatial dimensions of the output are fully determined by the task before any architecture decisions are made.

(b)

With padding 1 and stride 1, a  $3 \times 3$  convolution preserves spatial dimensions:  $H_{out} = H_{in}$ . Pooling:  $H_{out} = H_{in}/2$ . Unpooling:  $H_{out} = H_{in} \times 2$ . Pooling layers have no trainable parameters. The convolution layers have  $(C_{in}M^2 + 1)C_{out}$  parameters.

| Layer    | Output dimensions | Total params |
|----------|-------------------|--------------|
| CONV-1   | $(32, 32, 8)$     | 224          |
| POOL-1   | $(16, 16, 8)$     | 0            |
| CONV-2   | $(16, 16, 16)$    | 1168         |
| POOL-2   | $(8, 8, 16)$      | 0            |
| CONV-BN  | $(8, 8, 32)$      | 4640         |
| UNPOOL-1 | $(16, 16, 32)$    | 0            |
| CONV-3   | $(16, 16, 16)$    | 4624         |
| UNPOOL-2 | $(32, 32, 16)$    | 0            |
| CONV-4   | $(32, 32, 8)$     | 1160         |

| Layer    | Output dimensions | Total params |
|----------|-------------------|--------------|
| CONV-OUT | (32, 32, 4)       | 36           |

(c) Total trainable parameters:

$$224 + 1168 + 4640 + 4624 + 1160 + 36 = \mathbf{11852}$$

The dominant layers are **CONV-BN** and **CONV-3**, each with over 4,600 parameters—together accounting for nearly 80% of all weights. Both are  $3 \times 3$  convolutions at the highest channel count near the bottleneck.

**Comparison with a fully connected layer.** A single FC layer mapping the input directly to the output would take a flattened input of size  $32 \times 32 \times 3 = 3,072$  and produce an output of size  $32 \times 32 \times 4 = 4,096$ , requiring:

$$(3072 + 1) \times 4096 = 12586496 \text{ parameters}$$

The entire encoder-decoder network uses only 11,852 parameters—roughly **1,000 times fewer** than a single FC layer performing the same mapping. This dramatic difference arises from weight sharing: convolutional filters are reused at every spatial position, making CNNs far more parameter-efficient for spatial tasks.

## 4.2 Automatic Differentiation

(a) The intermediate and output nodes implement the forward computation step by step:

$$v_5 = v_1 \times v_2 = xw \quad (\text{linear pre-activation, weight term})$$

$$v_6 = v_5 + v_3 = xw + b \quad (\text{linear pre-activation, full})$$

$$v_7 = \sigma(v_6) = y \quad (\text{sigmoid activation, network output})$$

$$v_8 = v_7 - v_4 = y - t \quad (\text{residual: prediction minus target})$$

$$v_9 = v_8^2 = (y - t)^2 = \mathcal{L} \quad (\text{loss: squared residual})$$

(b) Forward pass:

$$v_5 = 1 \times 2 = 2,$$

$$v_6 = 2 + (-2) = 0$$

$$v_7 = \sigma(0) = \frac{1}{1 + e^0} = \frac{1}{2} = 0.5$$

$$v_8 = 0.5 - 1 = -0.5,$$

$$v_9 = (-0.5)^2 = 0.25$$



(c) Backward pass, starting from  $\bar{v}_9 = 1$  and applying the reverse recursion  $\bar{v}_i = \sum_{j \in \text{ch}(i)} \bar{v}_j \frac{\partial v_j}{\partial v_i}$ :

$$\bar{v}_8 = \bar{v}_9 \cdot \frac{\partial v_9}{\partial v_8} = 1 \times 2v_8 = 2 \times (-0.5) = -1$$

$$\frac{\partial \mathcal{L}}{\partial y} = \bar{v}_7 = \bar{v}_8 \cdot \frac{\partial v_8}{\partial v_7} = \bar{v}_8 \times 1 = -1$$

$$\frac{\partial \mathcal{L}}{\partial (wx + b)} = \bar{v}_6 = \bar{v}_7 \cdot \sigma'(v_6) = \bar{v}_7 \cdot \sigma(v_6)(1 - \sigma(v_6)) = -1 \times 0.5 \times 0.5 = -\mathbf{0.25}$$

$$\bar{v}_5 = \bar{v}_6 \cdot \frac{\partial v_6}{\partial v_5} = \bar{v}_6 \times 1 = -0.25$$

$$\bar{v}_3 = \bar{v}_6 \cdot \frac{\partial v_6}{\partial v_3} = \bar{v}_6 \times 1 = -0.25$$

$$\frac{\partial \mathcal{L}}{\partial w} = \bar{v}_2 = \bar{v}_5 \cdot \frac{\partial v_5}{\partial v_2} = \bar{v}_5 \times v_1 = -0.25 \times 2 = -\mathbf{0.5}$$

$$\bar{v}_1 = \bar{v}_5 \cdot \frac{\partial v_5}{\partial v_1} = \bar{v}_5 \times v_2 = -0.25 \times 1 = -0.25$$

The gradient  $\partial \mathcal{L} / \partial (wx + b) = -0.25$  is attenuated by the factor  $\sigma'(v_6) = \sigma(0)(1 - \sigma(0)) = 0.25$ . This is the *maximum* value of the sigmoid derivative—it is achieved at  $w x + b = 0$ , exactly the case here. For any other input,  $|\sigma'(a)| < 0.25$ , and as  $|a| \rightarrow \infty$  the derivative approaches zero (saturation).

In a deep network with  $L$  layers, each using sigmoid activations, the gradient of the loss with respect to early-layer parameters is multiplied by a factor of at most  $0.25^L$  — shrinking exponentially with depth. This is the **vanishing gradient** problem, and is the main reason sigmoid activations were replaced by ReLU in deep networks: ReLU's derivative is either 0 or 1, never attenuating a flowing gradient